UNIVERSIDAD RAFAEL NÚÑEZ

Especialización en Analítica de Datos · Tecnologías de Procesamiento de Datos

El caso por lotes y en flujo

Sesión 2 · Sábado 31 de octubre · 8:00 a. m.–4:00 p. m. · en GitHub Codespaces

Docente: Heyder Medrano Olier

> *Ayer vimos por qué una sola máquina no alcanza. Hoy lo resuelven ustedes con el mes completo de viajes: en la mañana por lotes, en la tarde en tiempo real.*

Mañana · Unidad 1 · Lotes **11:00 · Proyecto Final** **Tarde · Unidad 3 · Tiempo real**

Así será el día: una simulación

Lo que van a escribir y lo que van a ver en su codespace, de las 8:00 a las 4:00. Las salidas son las reales de la prueba del 29 de octubre

- **8:00** Preparación Previa
- **8:20** ¿Basta un computador?
- **8:40** El mes entra a HDFS
- **8:55** El job MapReduce
- **9:10** Optimizar el job
- **9:45** Spark
- **11:00** Proyecto Final y teoría
- **1:30** Kafka y la app
- **2:00** Medir, caída y exactitud
- **2:50** Spark Streaming
- **3:45** Cierre y apagado

tpd-codespace [Codespaces] · Terminal — 8:00 — Preparación Previa (PDF)

```
▸ Abren el enlace de la guía y pulsan «Create new codespace»
⏳ el codespace se crea solo · unos 7 min
▸ Se abre VS Code en el navegador; aceptan «Confiar»
@tu-usuario ➜ /workspaces/tpd-codespace (main) $ bash tpd/preparar_todo.sh
⏳ descarga los datos y revisa todo · 5 min 19 s en la prueba
…
  [OK]    RAM: 15.6 GB
  [OK]    Disco libre: 11 GB

2. Docker
  [OK]    Docker encendido (motor 28.1.1-1)
  [OK]    Imagen apache/hadoop:3.4.3
  [OK]    Imagen spark:3.5.9-python3
  [OK]    Imagen confluentinc/cp-kafka:7.6.0
```

Qué hacen

Abren el enlace de la guía, crean su codespace y pegan un solo comando.

Qué deben ver

La última línea: «RESULTADO: LISTO para el Taller 1».

A la bitácora

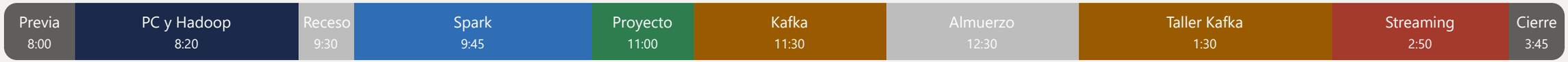
Nada todavía. Si no llega a LISTO, trabaja en el codespace de un compañero de su grupo.

▶ Reproducir | ← Etapa | Etapa → | ↺ Desde el inicio | Velocidad normal

Cada comando se copia de la guía, un recuadro a la vez. Donde la máquina trabaja varios minutos, la simulación lo resume en una barra; en clase, ese tiempo es para leer la guía y anotar.

Agenda del día

La mañana, por lotes; la tarde, en flujo. El mismo caso, todo en su codespace



8:00–9:30 · Codespace y Hadoop

Preparación Previa todos juntos; Excel y pandas frente al mes; MapReduce con 31 bloques y el combiner.

9:45–11:00 · Spark

El mismo cálculo con CSV y con Parquet, la Spark UI y la curva de núcleos.

11:00–12:30 · Proyecto y tiempo real

Definición del Proyecto Final; del lote al flujo; Kafka por dentro.

1:30–3:45 · El flujo en vivo

La app de viajes sobre Kafka; medir, tumbar y verificar; Spark Streaming.

Un problema por bloque y su solución con evidencia: todo lo de hoy corre en su codespace; el 6 y el 7 de noviembre, el mismo caso en la nube.

Cómo se trabaja hoy

Cinco acuerdos para que el día rinda: la máquina hace el trabajo pesado, ustedes miden e interpretan

📄 La guía manda

Cada comando está en su recuadro: se copia, se pega en la terminal y se compara con la salida que muestra la guía. Si algo falla, la guía dice qué hacer en ese mismo paso.

⏳ Mientras la máquina trabaja

Copiar el mes a HDFS tarda unos 4 minutos y cada job, unos 3. Ese tiempo es para leer el «Qué responde para el análisis» de la guía, no para escribir en la misma terminal.

📓 La bitácora

Tiempos, bloques, bytes de shuffle, eventos por segundo, offsets: cada cifra que anotan hoy es evidencia para comparar tecnologías y para el Proyecto Final.

👥 El grupo de 3

Si a alguien le falla el codespace, sigue en el de un compañero de su grupo. Nadie se queda atrás por una falla técnica.

💻 Cuidar las horas

La cuenta gratuita de GitHub trae 120 horas de cómputo al mes y la máquina de 4 núcleos gasta 4 por hora: unas 30 horas de uso. Al terminar, se detiene el codespace.

🎯 Qué se evalúa hoy

El taller de tiempo real se revisa con una lista de chequeo por etapa, y a las 11:00 cada equipo deja definido su Proyecto Final.

Su papel es el de analista: medir, comparar y decidir con cifras propias. Nadie tiene que programar.

Fuentes: GitHub (s. f.), facturación de Codespaces · 📚 Bibliografía

8:00 · Preparación Previa, todos juntos

Un computador en la nube para cada uno: 4 núcleos y 16 GB, con Hadoop, Spark y Kafka listos para usar

1. Entrar con su cuenta

La cuenta de GitHub que crearon ayer, con el correo verificado.

2. Crear el codespace

El enlace de la guía y el botón «Create new codespace». Se crea solo en unos 7 minutos.

3. Pegar un comando

`bash tpd/preparar_todo.sh` descarga los datos y revisa el equipo: unos 5 minutos.

4. Ver «LISTO»

La última línea dice «RESULTADO: LISTO para el Taller 1». Con eso, el día está asegurado.

Qué es un codespace

Un computador Linux que GitHub presta en la nube y que se usa desde el navegador. Por dentro corre Docker: Hadoop, Spark y Kafka viven en contenedores, con las mismas versiones oficiales que usan las empresas. No se instala nada en el computador de cada uno.

Si algo falla

Si el comando termina con avisos, la guía dice qué hacer con cada uno. Si a las 8:20 alguien no llegó a LISTO, trabaja en el codespace de un compañero de su grupo y lo resolvemos en el receso.

Mientras se crean los codespaces, vemos la simulación del día: así nadie espera sin saber qué viene.

Fuentes: GitHub (s. f.), facturación de Codespaces · 📚 Bibliografía

¿Por qué no basta un computador?

8:20–8:40 · Taller 1A, Parte A: el mes de junio de 2025 frente a Excel y pandas

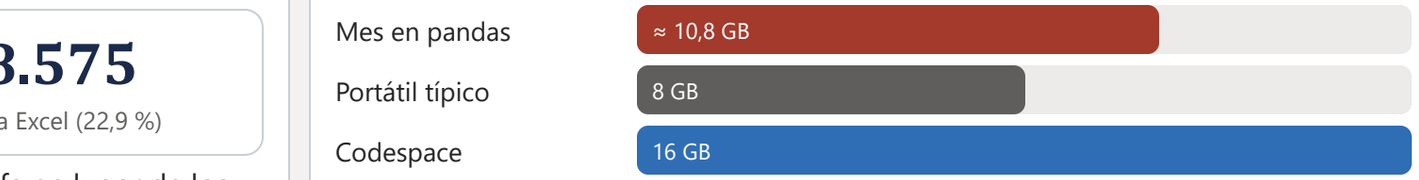
Excel: el mes (4 GB) no abre; la semana no cabe

4.576.158
viajes de la semana

1.048.575
viajes que carga Excel (22,9 %)

Llega hasta el 3 de junio a las 21:04: US$ 28,1 millones de tarifa en lugar de los US$ 132,1 millones de la semana.

pandas: memoria para el mes completo



Medido en el codespace: 0,54 GB por millón de viajes. El sistema ya usa parte de la RAM.

Límite de la herramienta

Excel admite 1.048.576 filas por hoja y no avisa en las celdas que faltan datos: el total sale incompleto y parece correcto.

Límite de la máquina

La memoria de un solo equipo: un año de viajes pediría doce veces más.

Con un solo computador

La gerencia pide el informe del mes y recibe un informe incompleto que parece correcto, o un proceso que no termina.

Repartiendo el trabajo

El archivo en bloques y el cálculo en varias tareas: lo que hace Hadoop desde las 8:40.

El problema no es la herramienta: el dato creció más que la máquina.

Fuentes: Microsoft (s. f.), límites de Excel; TLC (2025) · Bibliografía

Hadoop con el mes completo: del archivo a dos líneas

8:40–9:30 · Taller 1A, Parte B: lo que deben encontrar en las consolas

CSV 4,04 GB
tpd/datos del codespace
→
HDFS
31 bloques de 128 MB
→
31 tareas map
2 a la vez (YARN: 4.096 MB)
→
Shuffle
19,9 M pares · 370 MB
→
1 reducer
suma por empresa
→
Salida
2 líneas: Uber y Lyft

Cómo leer los contadores del job

- **31 splits = 31 bloques = 31 maps:** el paralelismo lo decide el tamaño del archivo.
- **Map input 19.868.010:** los viajes más el encabezado.
- **Reduce shuffle bytes 369.900.781:** lo que viajó de los maps al reducer.
- **Elapsed 3 min 33 s:** cada map tarda 6 s, pero van por tandas de 2.

El resultado: la pregunta 1 del caso

Empresa	Viajes	Tarifa base US$	Pago conductor US$	Plataforma
Uber	14.167.361	411.920.420,29	316.067.761,82	23,3 %
Lyft	5.700.648	145.003.586,84	113.817.449,39	21,5 %

Lo que hace un año exigía un clúster, aquí sale en unos 3 minutos con el archivo completo, sin muestras.

Hadoop reparte el archivo (HDFS), asigna memoria y núcleos (YARN) y calcula por partes (MapReduce). Hoy lo ven funcionando y lo miden.

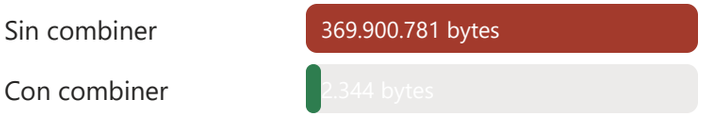
Fuentes: Dean y Ghemawat (2008); Shvachko et al. (2010); Vavilapalli et al. (2013) · Bibliografía

Optimizar: combiner, memoria y su propio job

Taller 1A, Parte B, pasos 6 a 8: tres maneras de mejorar un job y una pregunta nueva del caso

Paso 6 · El combiner

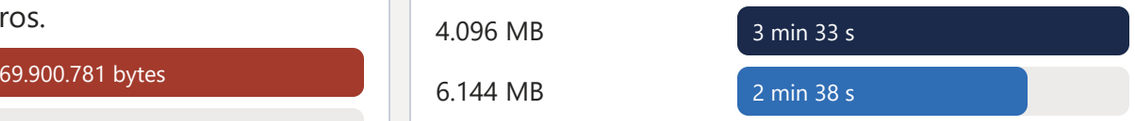
El reducer suma dentro de cada map antes del shuffle (sumar es asociativo): cada map envía 2 líneas, 31 × 2 = 62 registros.



Resultado idéntico al centavo; el tiempo baja de 3 min 33 s a 3 min 14 s.

Paso 7 · Más memoria para YARN (RA4)

Con 4.096 MB caben 2 tareas a la vez; con 6.144 MB, 4: una por núcleo.



Configurar es esto: cambiar un parámetro y demostrar con el tiempo que mejoró (un 26 % aquí).

Paso 8 · Su propio job: la espera por zona

Antes de ver el código, cada equipo diseña la clave, el valor y por qué se suman espera y conteo por separado.

Zona	Viajes	Espera
Flushing Meadows-Corona Park	23.764	10,84 min
Hammels/Arverne	32.624	8,27 min
Far Rockaway	43.788	7,95 min

El job descarta 231.332 esperas negativas: calidad del dato.

Error común

Creer que el combiner acelera mucho aquí. En un solo equipo pesa más la lectura; en un clúster real, el shuffle cruza la red y ahí sí cambia todo.

¿Y un promedio?

Promediar promedios da un resultado falso. Se suman por separado la espera y el conteo, y se divide al final: así se hizo el paso 8.

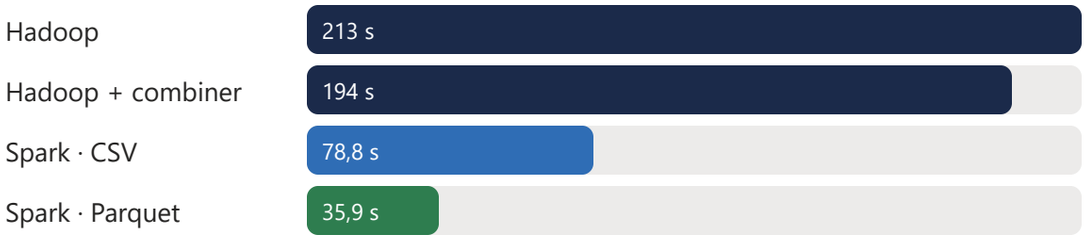
Optimizar es mover menos datos y usar mejor la máquina, y demostrarlo con cifras.

Fuentes: Dean y Ghemawat (2008); White (2015) · Bibliografía

Spark frente a MapReduce: el mismo resultado, otro camino

9:45–11:00 · Taller 1A, Parte C: tiempos y datos leídos, medidos en el mismo codespace

Tiempo del mismo cálculo (segundos)



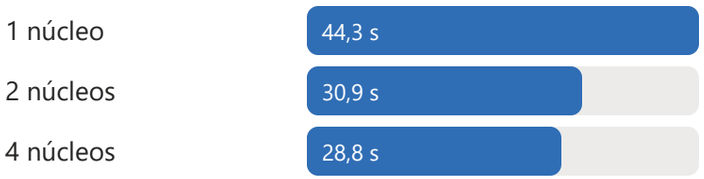
Lo que se leyó y lo que se movió

	Tareas de lectura	Datos leídos	Shuffle
CSV	31	3,8 GiB	6,9 KiB
Parquet	4	70,8 MiB	955 B

Agregación parcial: Spark suma dentro de cada tarea antes del shuffle, como un combiner automático.

Formato: Parquet guarda por columnas y comprimido: Spark leyó solo las 3 columnas del cálculo, unas 55 veces menos.

Paso 6 · ¿Más núcleos, la mitad del tiempo?



No: hay una parte fija de arranque y el Parquet solo ofrece 4 tareas de lectura.

Honestidad de la comparación

Spark usó los 4 núcleos sin el límite de memoria de YARN y leyó del disco del codespace, no de HDFS: no es una carrera igual. Los tiempos cambian entre corridas: se reporta lo medido, no un número fijo.

Lo que sí se concluye

Menos disco entre fases y menos datos leídos: ahí está la ganancia. Elegir motor y formato es una decisión de rendimiento y de costo, con evidencia de la Spark UI.

Mismo resultado al centavo en los cuatro casos; cambia cuánto se lee, cuánto se mueve y cuánto se tarda.

Fuentes: Zaharia et al. (2012); Apache Parquet (s. f.) · Bibliografía

11:00 · Cierre de la Unidad 1: el Proyecto Final

11:00–11:30 · Se define con la evidencia de la mañana, no como una tarea aparte

1
La necesidad
Qué decisión del caso de viajes resuelve su arquitectura y para quién: gerencia, operación o regulador.

2
La arquitectura
Hadoop, Spark o híbrida, en el codespace y en la nube; qué formato de datos y por qué.

3
La justificación
Tiempos, contadores y bytes medidos hoy; qué cambia con un año de datos.

4
La sustentación
Sábado 7 de noviembre: configuración, integración de datos y rendimiento, según la rúbrica.

Dónde está todo
El enunciado, las ocho preguntas, los entregables y la rúbrica están en la carpeta de la Sesión 2 («Proyecto Final», PDF). Los programas del proyecto ya están en el codespace.

Cómo se califica
Diseño y configuración de un sistema distribuido: lo que se evalúa es la decisión y su evidencia, no la cantidad de código.

Pregunta a cada equipo antes de cerrar: ¿en qué escenario del caso elegirían Hadoop puro sobre Spark? Un caso concreto, no una respuesta genérica.

UNIDAD 3 · 11:30 a. m.–12:30 p. m.

Procesamiento de datos en tiempo real

Conceptos y Apache Kafka; en la tarde, el taller con la app de viajes

> *¿Qué decisión del caso no puede esperar al proceso de las 2 de la mañana?*

Mañana · Lotes | Proyecto Final | **Tarde · Tiempo real**

Del lote al flujo: la misma cadena, otra urgencia

Lo que hicieron en la mañana responde mañana; lo de la tarde responde en segundos

Por lotes (la mañana)

Viajes del día → HDFS → Job nocturno → Informe

Datos ya acumulados · responde en horas · la pregunta: ¿qué pasó?

En flujo (la tarde)

Evento del viaje → Kafka → Spark Streaming → Decisión

Cada evento a medida que llega · responde en segundos · la pregunta: ¿qué está pasando ahora? (en el taller: latencia de 2 a 4 ms en Kafka)

Lote

Se ejecuta cada cierto tiempo sobre datos acumulados: barato y completo, pero tarde.

Flujo

Procesa cada evento al llegar: exige que ningún evento se pierda y que el sistema no se detenga.

Escenario: 7:58 p. m., llueve en Midtown. Suben las solicitudes y bajan los carros libres. El informe de mañana lo confirmará; la tarifa dinámica debe reaccionar ahora.

Si solo hay lotes

La empresa se entera mañana de que perdió viajes esta noche.

Con flujo y lotes

El flujo decide ahora; el lote consolida y audita después. Los dos leen los mismos eventos.

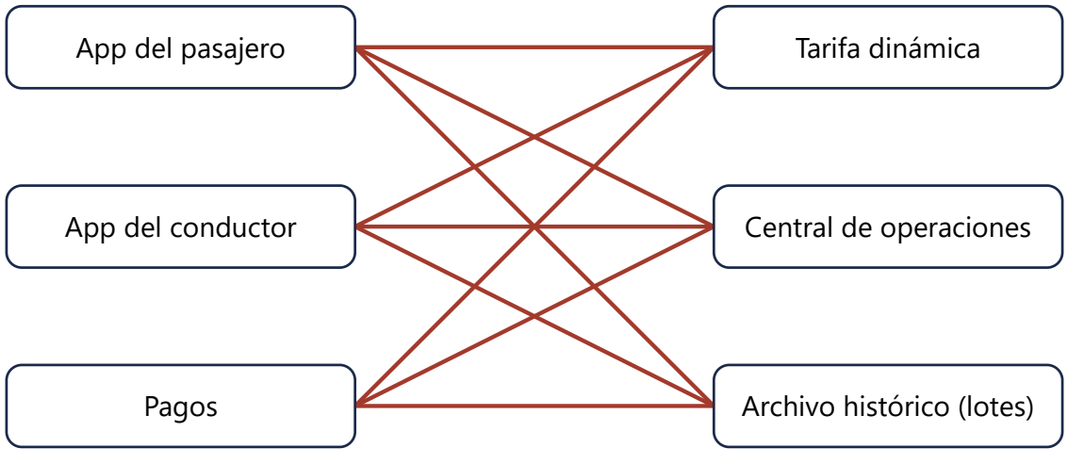
Kafka es el punto de encuentro: publica cada evento una vez y lo entrega al flujo y al lote.

Fuentes: Kleppmann (2017), cap. 11; Akidau et al. (2015) · 📚 Bibliografía

Por qué Kafka: el modelo publicación/suscripción

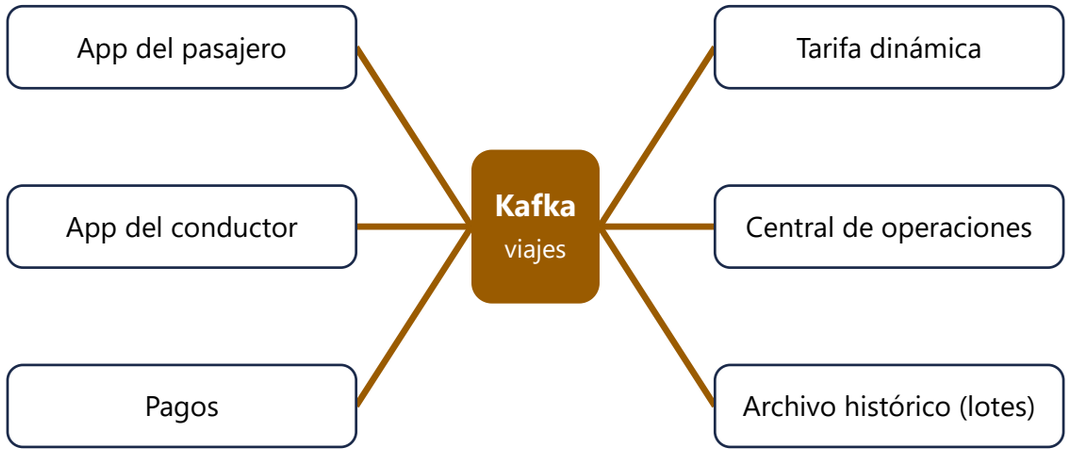
De conectar cada sistema con cada otro a publicar una vez y leer muchas

Sin intermediario: conexiones punto a punto



3 × 3 = 9 integraciones que mantener; si un sistema cae, arrastra a otros.

Con Kafka: productores y consumidores desacoplados



3 + 3 = 6 conexiones; nadie espera a nadie.

Desacoplamiento
La app publica en un tema sin saber quién ni cuántos leerán.

Cada consumidor a su ritmo
La tarifa dinámica lee en milisegundos; el archivo histórico puede leer cada hora.

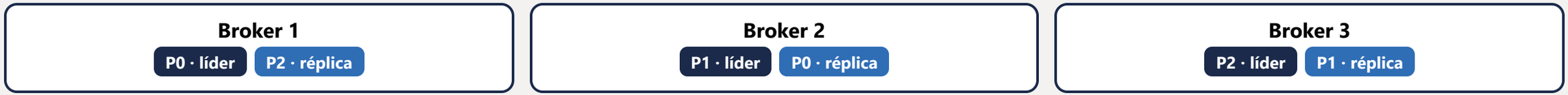
Los eventos quedan guardados
Un consumidor nuevo puede leer también lo que ya se publicó.

Kafka no es una base de datos ni una cola que se vacía: es un registro de eventos al que muchos se suscriben. Uber lo usa en su infraestructura de tiempo real.

Fuentes: Kreps et al. (2011); Fu y Soman (2021) · Bibliografía

Brokers, réplicas y líder: qué pasa si un servidor cae

Tema «viajes» con 3 particiones y factor de replicación 2. Haz clic en un broker para apagarlo



Los 3 brokers funcionan: cada partición tiene su líder (azul oscuro) y una réplica en otro broker (azul claro).

Broker
Cada servidor del clúster de Kafka.

Partición
Divide el tema para trabajar en paralelo. Todos los eventos de un viaje van a la misma partición (la clave es el viaje): así llegan en orden.

Réplica
Copia de una partición en otro broker; el factor de replicación dice cuántas hay. El líder atiende escrituras y lecturas.

En el taller de esta tarde 1 broker, 2 particiones y réplica 1. Sirve para ver la mecánica, pero no tolera fallos: en producción se usan al menos 3 brokers.

Partición ≠ réplica. Las particiones reparten el trabajo; las réplicas protegen contra la caída de un servidor.

Fuentes: Apache Kafka (s. f.); Kleppmann (2017), caps. 5 y 6 · Bibliografía

Garantías de escritura y retención

Cuánto riesgo de pérdida acepta cada evento del viaje, y por qué leer no es borrar

acks = 0

Cuándo confirma: nunca: el productor envía y sigue sin esperar respuesta.

Riesgo: Puede perder mensajes sin enterarse.

Uso típico: La ubicación del conductor cada pocos segundos: perder una no cambia la decisión.

acks = 1

Cuándo confirma: cuando el líder escribió el mensaje.

Riesgo: Si el líder cae antes de replicar, el mensaje se pierde.

Uso típico: Registros de la aplicación.

acks = all

Cuándo confirma: cuando todas las réplicas sincronizadas lo tienen.

Riesgo: Más latencia, máxima durabilidad.

Uso típico: El pago del viaje. Es el valor por defecto del productor desde Kafka 3.0.

Retención: Kafka no borra un mensaje cuando alguien lo lee. Partición P0: cada mensaje tiene un offset.



▶ El consumidor caído vuelve ↺ Reiniciar

> Consumidor caído: último leído 3. Consumidor al día: último leído 8. Los mensajes 4 a 9 siguen guardados.

Los mensajes se conservan por tiempo o por tamaño: por defecto, 7 días.

Si el consumidor vuelve después del plazo de retención, esos mensajes ya no existen.

Fuentes: Apache Kafka (s. f.); Apache Software Foundation (s. f.), KIP-679 · Bibliografía

Taller: los viajes en tiempo real sobre Kafka

1:30–2:50 · Taller 1B, Parte A: qué deben observar



Un viaje = 6 eventos (solicitud, asignado, llegada, recogida, fin, pago) con la misma clave: misma partición, en orden.

Lag: lo que le falta por leer a un grupo de consumidores; si crece sin parar, el consumidor no da abasto.

Escenario: la central se reinicia en plena hora pico. Mientras está caída, los eventos siguen llegando a Kafka y quedan guardados.

Sin offsets confirmados

La central empezaría desde cero o desde el final: repetiría o perdería viajes.

Con grupo de consumidores

Retoma exactamente donde quedó: ni pierde ni repite. En la prueba: 1.000 viajes publicados, 0 duplicados, montos idénticos al centavo, «RESULTADO: PRECISO».

Evidencia a registrar: throughput, latencia, offset de reinicio, lag y el resultado de la verificación.

Fuentes: Kreps et al. (2011); Apache Kafka (s. f.); datos: TLC (2025) · Bibliografía

Spark Streaming: ventanas y watermark

2:50–3:45 · Taller 1B, Parte B: la espera por zona, mientras ocurre

Ventanas de 15 minutos sobre la hora del viaje

00:00 · cerrada	00:15 · abierta	00:30 · abierta	00:45 · abierta

Cada 10 segundos (un lote) se actualizan las ventanas que cambiaron. **Evento más reciente visto:** 00:50. **Watermark = 30 min antes:** 00:20.

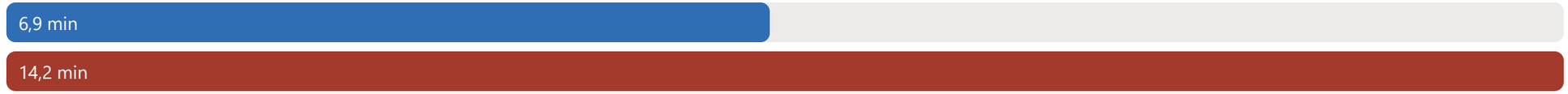
Llega tarde, dentro de los 30 min: se agrega a su ventana.

Más atrasado: su ventana ya se cerró y Spark liberó su memoria.

Ventana: agrupa por la hora en que ocurrió el viaje, no por la hora en que llegó el evento.

Watermark: cuánto atraso se acepta; sin él, un flujo que nunca termina guardaría todas las ventanas para siempre.

Escenario: LaGuardia a medianoche (datos reales del 2 de junio)



Señal

La espera sube en una zona con muchos viajes: faltan conductores.

Decisión

Subir la tarifa o dar incentivos a los conductores cercanos, ahora y no mañana.

El flujo convierte eventos en una señal accionable cada 10 segundos.

Fuentes: Armbrust et al. (2018); Akidau et al. (2015); Apache Spark (s. f.) · Bibliografía

Qué cambia en el análisis de datos

La tecnología de procesamiento decide qué preguntas puede responder el analista, con qué datos y a tiempo

	Sin tecnología de procesamiento	Con la evidencia de hoy
Completitud	Excel no abre el mes y de la semana analizó menos de 1 de cada 4 viajes, sin avisar.	El 100 % de los 19,9 millones de viajes.
Exactitud	US$ 28,1 millones de tarifa en vez de 132,1 de la semana: un total equivocado que parece correcto.	US$ 556,9 millones, idéntico al centavo en Hadoop y Spark.
Oportunidad	El informe por lotes responde al día siguiente.	La espera por zona se actualiza cada 10 segundos.
Escala	Un mes pide unos 11 GB en un solo equipo; un año, doce veces más.	Más bloques y más tareas: se agregan máquinas, no se cambia el análisis.
Costo	Leer 3,8 GiB de CSV para usar 3 columnas.	Parquet: 71 MiB leídos. En la nube se paga por lo que se lee.
Preguntas nuevas	¿Qué pasó el mes pasado?	¿Dónde faltan conductores ahora mismo y qué hacemos?

El analista no solo elige el modelo: elige, o debe exigir, la tecnología que hace posible un análisis completo, exacto, oportuno y a un costo razonable.

Cierre del día

Hoy, en GitHub Codespaces

Límite de un PC | HDFS y YARN | MapReduce y combiner | Spark y Parquet | Proyecto Final | Kafka | Offsets y lag | Ventanas y watermark

Próximo encuentro (6 y 7 de noviembre): el mismo caso en la nube

1. Preparación Previa del Encuentro 2: cuenta de Google Cloud (prueba gratuita o en pareja)

2. Clúster de Spark en Google Cloud con los datos en Cloud Storage

3. BigQuery y tablero en Looker Studio

4. Sustentación del Proyecto Final y del caso empresarial

Antes de irse: apaguen el codespace (F1 → «Codespaces: Stop Current Codespace»). Los datos quedan para el Proyecto Final y no se gastan horas.

Bibliografía

Las fuentes de cada concepto de la sesión (normas APA, 7.ª ed.). Cada enlace abre la fuente en una pestaña nueva

El caso y el entorno

New York City Taxi and Limousine Commission. (2025). *TLC trip record data: High volume for-hire vehicle trip records, junio de 2025* [Conjunto de datos]. https://www.nyc.gov/site/tlc/about/tlc-trip-record-data.page

Microsoft. (s. f.). *Excel specifications and limits*. Microsoft Support. https://support.microsoft.com/en-us/office/excel-specifications-and-limits-1672b34d-7043-467e-8e27-269d656771c3

GitHub. (s. f.). *GitHub Codespaces billing*. GitHub Docs. https://docs.github.com/en/billing/concepts/product-billing/github-codespaces

Mañana · Lotes

Dean, J., & Ghemawat, S. (2008). MapReduce: Simplified data processing on large clusters. *Communications of the ACM, 51*(1), 107–113. https://doi.org/10.1145/1327452.1327492

Shvachko, K., Kuang, H., Radia, S., & Chansler, R. (2010). The Hadoop distributed file system. En *2010 IEEE 26th Symposium on Mass Storage Systems and Technologies (MSST)* (pp. 1–10). IEEE. https://doi.org/10.1109/MSST.2010.5496972

Vavilapalli, V. K., Murthy, A. C., Douglas, C., Agarwal, S., Konar, M., Evans, R., Graves, T., Lowe, J., Shah, H., Seth, S., Saha, B., Curino, C., O'Malley, O., Radia, S., Reed, B., & Baldeschwieler, E. (2013). Apache Hadoop YARN: Yet another resource negotiator. En *Proceedings of the 4th Annual Symposium on Cloud Computing* (pp. 1–16). ACM. https://doi.org/10.1145/2523616.2523633

Zaharia, M., Chowdhury, M., Das, T., Dave, A., Ma, J., McCauley, M., Franklin, M. J., Shenker, S., & Stoica, I. (2012). Resilient distributed datasets: A fault-tolerant abstraction for in-memory cluster computing. En *9th USENIX Symposium on Networked Systems Design and Implementation (NSDI 12)* (pp. 15–28). USENIX. https://www.usenix.org/conference/nsdi12/technical-sessions/presentation/zaharia

The Apache Software Foundation. (s. f.). *Apache Parquet documentation*. https://parquet.apache.org/docs/

Tarde · Tiempo real

Kreps, J., Narkhede, N., & Rao, J. (2011). Kafka: A distributed messaging system for log processing. En *Proceedings of the 6th International Workshop on Networking Meets Databases (NetDB)*.

Fu, Y., & Soman, C. (2021). Real-time data infrastructure at Uber. En *Proceedings of the 2021 International Conference on Management of Data (SIGMOD '21)* (pp. 2503–2516). ACM. https://doi.org/10.1145/3448016.3457552

The Apache Software Foundation. (s. f.). *Apache Kafka documentation*. https://kafka.apache.org/documentation/

The Apache Software Foundation. (s. f.). *KIP-679: Producer will enable the strongest delivery guarantee by default*. Apache Kafka. https://cwiki.apache.org/confluence/display/KAFKA/KIP-679%3A+Producer+will+enable+the+strongest+delivery+guarantee+by+default

Armbrust, M., Das, T., Torres, J., Yavuz, B., Zhu, S., Xin, R., Ghodsi, A., Stoica, I., & Zaharia, M. (2018). Structured streaming: A declarative API for real-time applications in Apache Spark. En *Proceedings of the 2018 International Conference on Management of Data (SIGMOD '18)* (pp. 601–613). ACM. https://doi.org/10.1145/3183713.3190664

Akidau, T., Bradshaw, R., Chambers, C., Chernyak, S., Fernández-Moctezuma, R. J., Lax, R., McVeety, S., Mills, D., Perry, F., Schmidt, E., & Whittle, S. (2015). The dataflow model: A practical approach to balancing correctness, latency, and cost in massive-scale, unbounded, out-of-order data processing. *Proceedings of the VLDB Endowment, 8*(12), 1792–1803. https://doi.org/10.14778/2824032.2824076

The Apache Software Foundation. (s. f.). *Structured Streaming programming guide (Spark 3.5.9)*. https://spark.apache.org/docs/3.5.9/structured-streaming-programming-guide.html

Libros de consulta

Kleppmann, M. (2017). *Designing data-intensive applications: The big ideas behind reliable, scalable, and maintainable systems*. O'Reilly Media. ISBN 978-1-4493-7332-0. https://www.oreilly.com/library/view/designing-data-intensive-applications/9781491903063/

White, T. (2015). *Hadoop: The definitive guide* (4.ª ed.). O'Reilly Media. ISBN 978-1-4919-0163-2. https://www.oreilly.com/library/view/hadoop-the-definitive/9781491901687/